

Resumo 4 - Arthur Choi Braga

Contexto e história

A orientação a objetos (OO) existe desde os anos 1960, mas ganhou força nos anos 1990 com a popularização da internet. Tecnologias OO se destacam em redes por transportar dados e comportamentos juntos. Conceitos OO são estáveis e relevantes até hoje, mesmo com a rápida evolução das tecnologias.

Procedural vs. Orientado a Objetos

No modelo procedural, dados e comportamentos são separados; os dados globais podem ser acessados e modificados por qualquer função, dificultando testes e manutenção. No modelo OO, dados (atributos) e comportamentos (métodos) são encapsulados em um único objeto, garantindo maior controle e integridade.

Objetos e Classes

- **Objetos:** entidade que combina atributos (dados/estado) e métodos (comportamentos). Comunica-se com outros objetos por meio de mensagens (chamadas métodos públicos).
- **Classes:** blueprint (molde) para criar objetos. Define os atributos e métodos que todos os objetos instanciados terão. Um objeto não pode existir sem uma classe.

Os 4 Pilares da OO

Os 4 pilares da orientação a objetos são: encapsulamento, herança, polimorfismo e composição.

Encapsulamento: agrupa atributos e métodos em um único objeto, controlando o acesso externo. Atributos são privados; o acesso ocorre por getters e setters (data hiding). Facilita manutenção, segurança e testes.

Herança: permite que uma classe herde atributos e métodos de outra (superclasse/pai). Evita duplicação de código. Relação "é um" (ex: cachorro é um mamífero). Pode ser simples (Java, C#) ou múltipla (C++).

Polimorfismo: objetos diferentes respondem de formas distintas à mesma mensagem. Ex: Circle, Square, Star recebem o mesmo comando draw(), mas cada um se desenha de forma diferente. Implementado por sobrescrita de métodos (override).

Composição: um objeto é construído contendo outros objetos. Relação "tem um" (ex: Carro tem um Motor). Alternativa à herança para reutilização de código, com menor acoplamento.

Interface vs. Implementação

A interface é o conjunto de métodos públicos que definem como o objeto se comunica com o mundo externo. A implementação é privada — pode ser alterada sem impactar quem usa o objeto. Analogia: uma tomada elétrica (interface) permite que qualquer aparelho use energia, sem conhecer como ela é gerada (implementação).

Sistemas Legados e OO

OO não substitui o modelo procedural — elas se complementam. Sistemas legados funcionais não devem ser reescritos sem motivo. Quando necessário usar obj. wrappers.